

Instituto Tecnológico de Buenos Aires



Trayecto de Tecnología - Secretaría de Admisión
Challenge Tecnológico Integrador: "GuardiánClima ITBA"
Documento del Proyecto

Grupo: 33

Integrantes:

- Juan Ignacio Bariffi Perez Chada (69056)
- Juan Mateo Beltran Flores (68928)
- Francisco Pedemonte (69099)
- Jerónimo Sanguinetti (68880)

Profesores:

- Javier Apat
- Nicolás Betti
- Flavio Fabian Espeche Nieva
- María Cecilia Novello

Fecha de Realización: 24/05/2026

Fecha de Entrega: 31/05/2026

Tabla de contenidos

1. Introducción.....	3
2. Diseño y Arquitectura.....	3
3. Guía de Usuario Detallada.....	10
4. Desafíos y Soluciones.....	10
5. Conclusiones y Aprendizajes del Equipo.....	11

1. Introducción

GuardiánClima ITBA (que en nuestra app llamamos "Windex") es un programa de consola en Python que permite consultar el clima de distintas ciudades, guardar un historial global de consultas, obtener estadísticas de uso y recibir un consejo de vestimenta generado por Inteligencia Artificial (Gemini). El objetivo fue integrar en una sola aplicación los temas de la materia: Programación (el desarrollo del código y el manejo de archivos), Ciberseguridad (validación de contraseñas), Análisis de Datos con Pandas (historial y estadísticas), Inteligencia Artificial (Google Gemini) y Cloud Computing y Conectividad (el uso de las APIs de OpenWeatherMap y Google Gemini).

La aplicación busca brindar al usuario una herramienta simple para acceder a información meteorológica en tiempo real y tomar decisiones en base a esos datos. Asimismo, incorpora mecanismos básicos de ciberseguridad mediante la validación de contraseñas, funcionalidades de análisis de datos a partir del historial de consultas y recomendaciones personalizadas generadas por IA. De esta manera, el proyecto permite aplicar de forma integrada conocimientos de distintas áreas tecnológicas en un entorno real.

Video De Introducción: https://youtu.be/SmK_DYuOei4

2. Diseño y Arquitectura

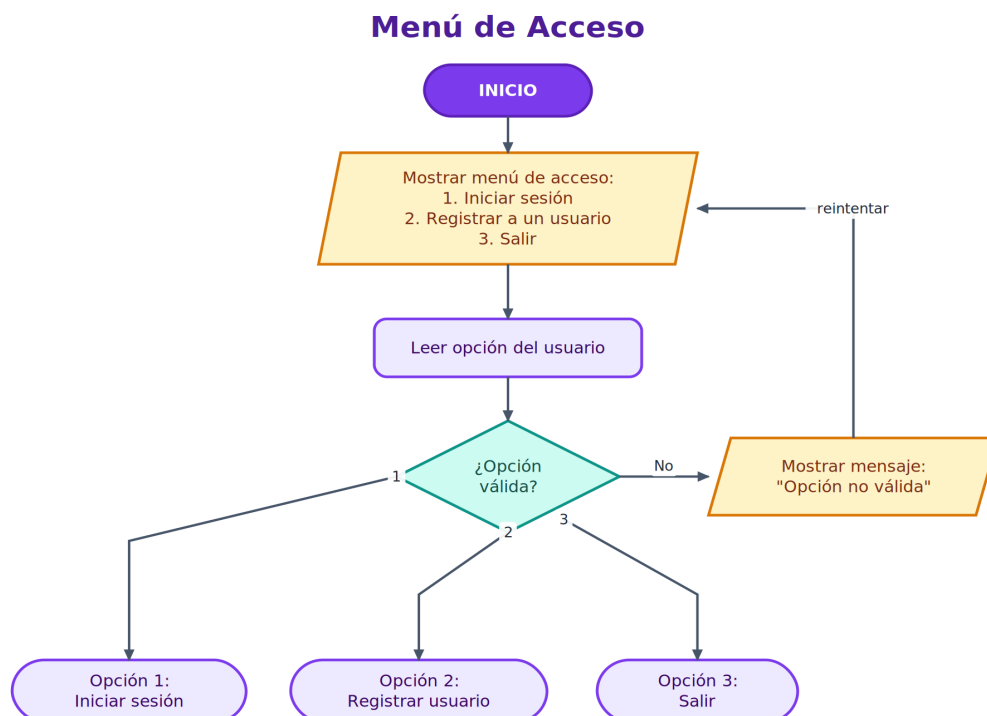
El programa se divide en módulos, cada uno con una responsabilidad clara:

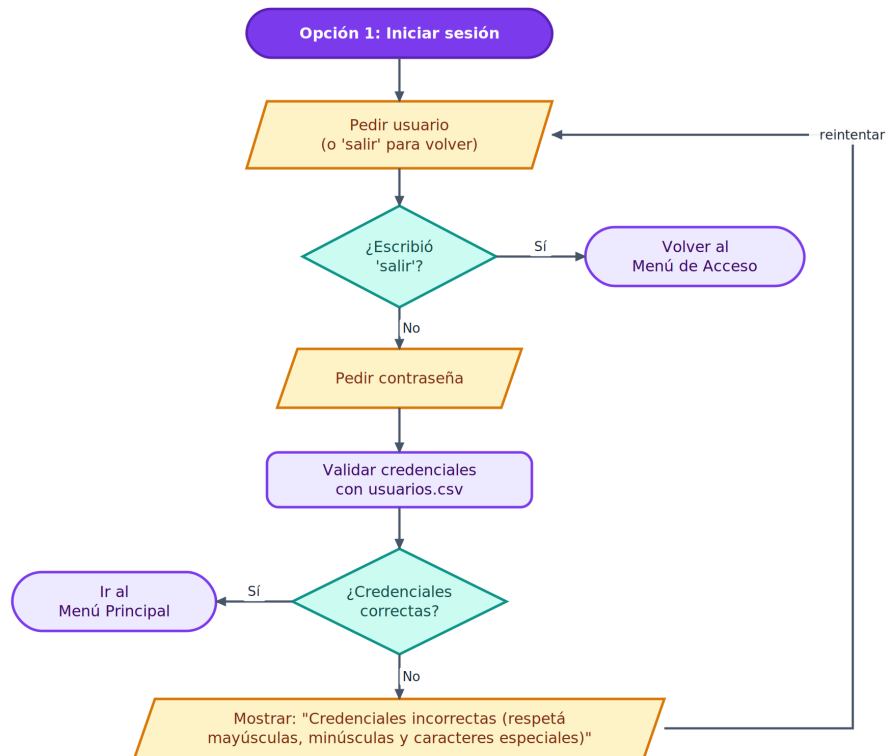
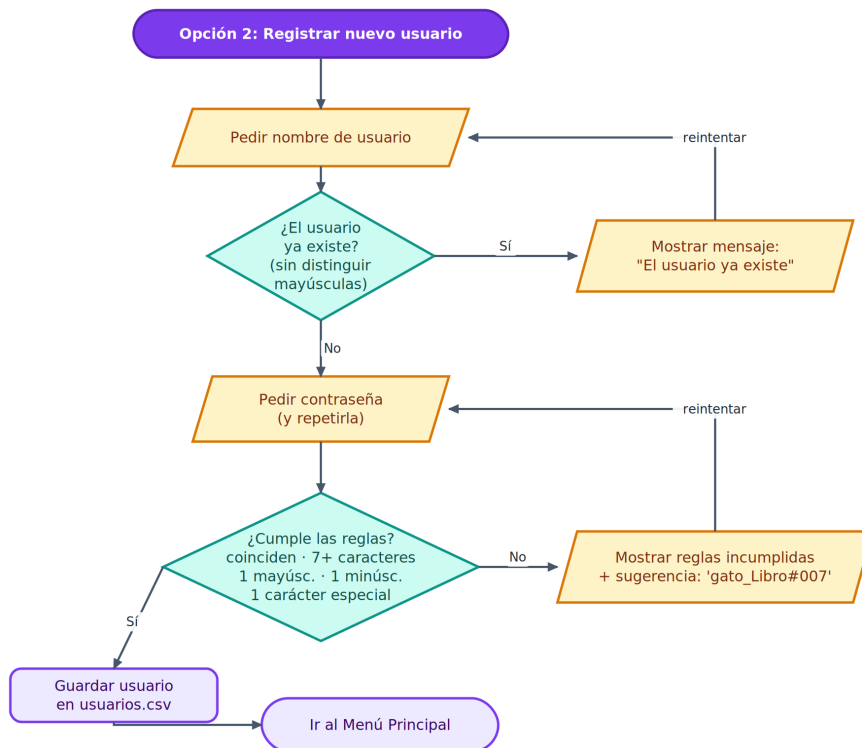
- **main.py**: punto de entrada y Menú de Acceso.
- **auth.py**: registro, login y validación de contraseñas.
- **menu_principal.py**: Menú Principal post-login y sus submenús.
- **clima.py**: conexión a OpenWeatherMap, guardado y muestra del historial en el archivo historial_global.csv.
- **estadisticas.py**: cálculo de estadísticas y exportación del historial.
- **asistente.py**: consejo de vestimenta con la IA de Gemini.

- **api.py**: módulo separado donde se almacenan las claves de API (OWM y Gemini), manteniéndose fuera de la lógica del programa.
- **limpiar_consola.py**: utilidad para limpiar la pantalla de la consola (no es necesaria para el funcionamiento de la aplicación pero emprolija su uso).

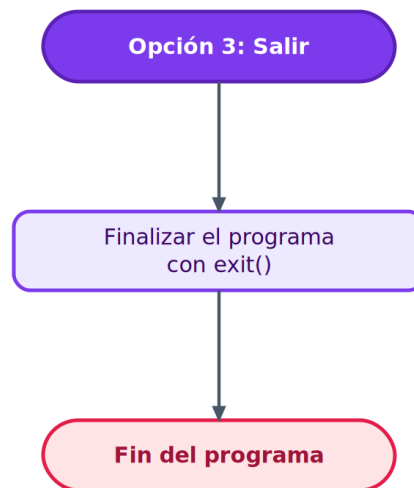
La app arranca en el Menú de Acceso y solo tras un login o registro exitoso se llega al Menú Principal. La lectura y escritura de archivos se hace con la librería pandas (Bloque 1.6.2). Los datos se guardan en dos archivos CSV: usuarios.csv (Usuario, Contraseña) e historial_global.csv (Usuario, Ciudad, Fecha, Temperatura, Descripción, Humedad, Viento).

Un dato de clima recorre el siguiente flujo: el usuario escribe una ciudad → se le pide el clima a OpenWeatherMap → la API responde en formato JSON → se extraen los valores, se muestran en pantalla y se guardan en el historial. Como decisiones de diseño: las claves de API se aíslan en un módulo aparte (api.py), separado del resto del código; contiene valores de ejemplo que cada usuario reemplaza por sus propias claves; las conexiones tienen manejo de errores diferenciado (401, 404 y errores de conexión); y todo el historial es global, lo que permite calcular estadísticas sobre las consultas de todos los usuarios.

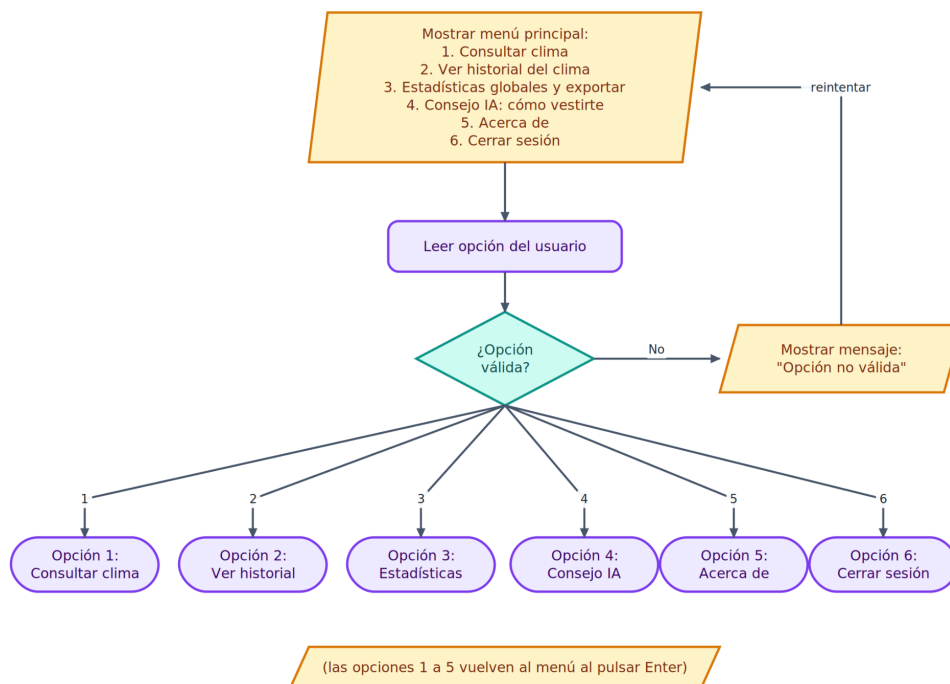


Opción 1: Iniciar sesión**Opción 2: Registrar nuevo usuario**

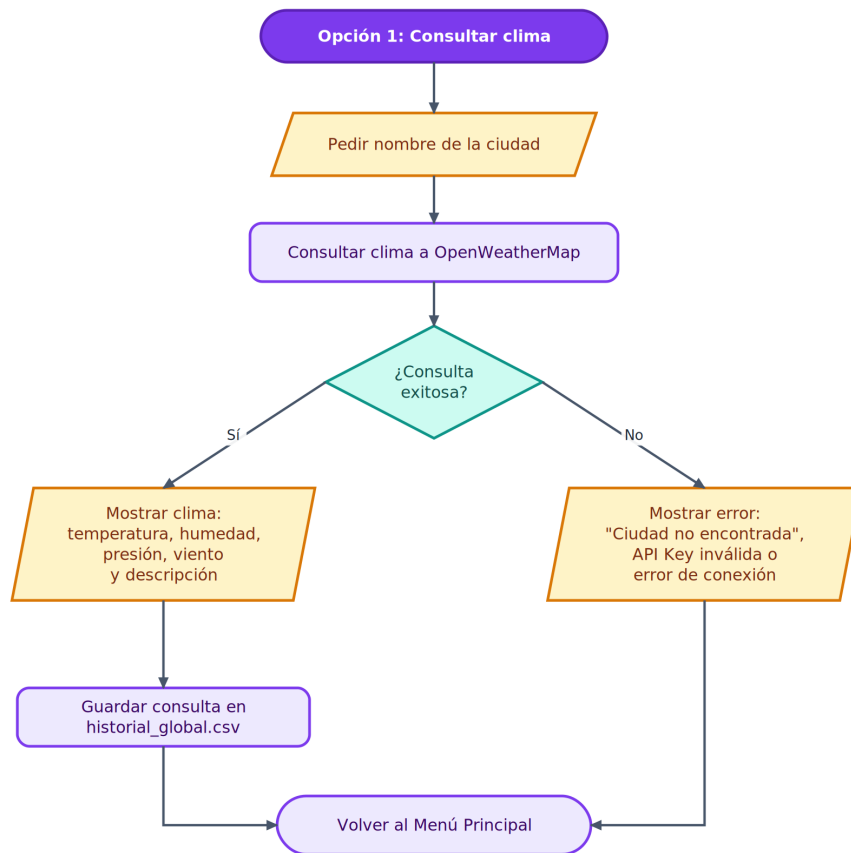
Opción 3: Salir



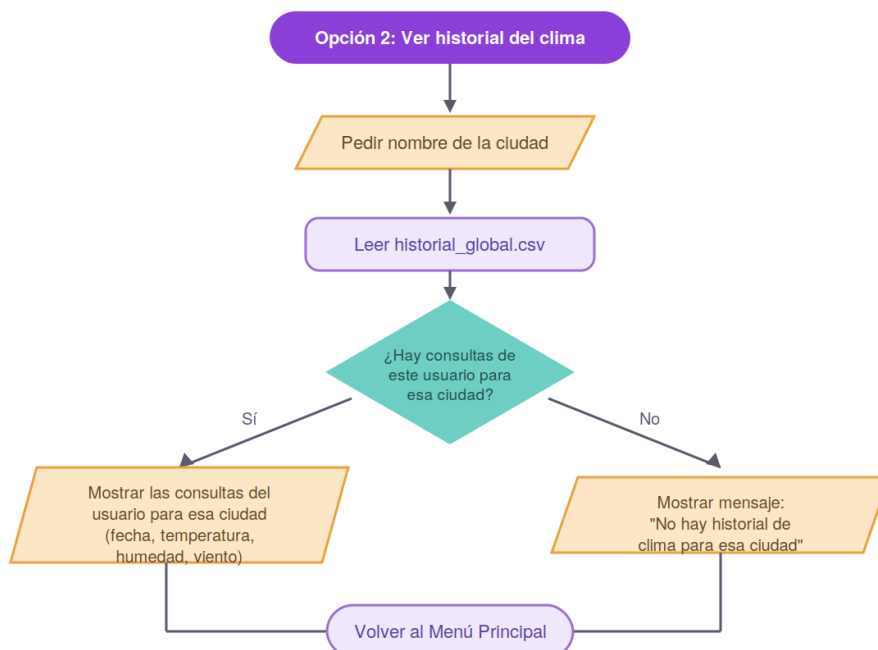
Menú Principal (post-login)



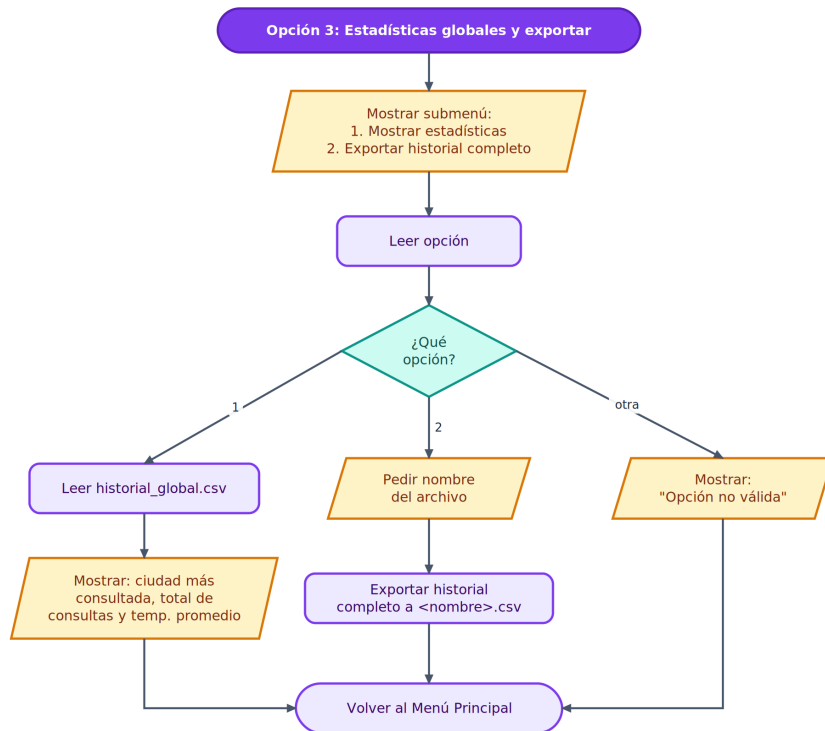
Opción 1: Consultar clima



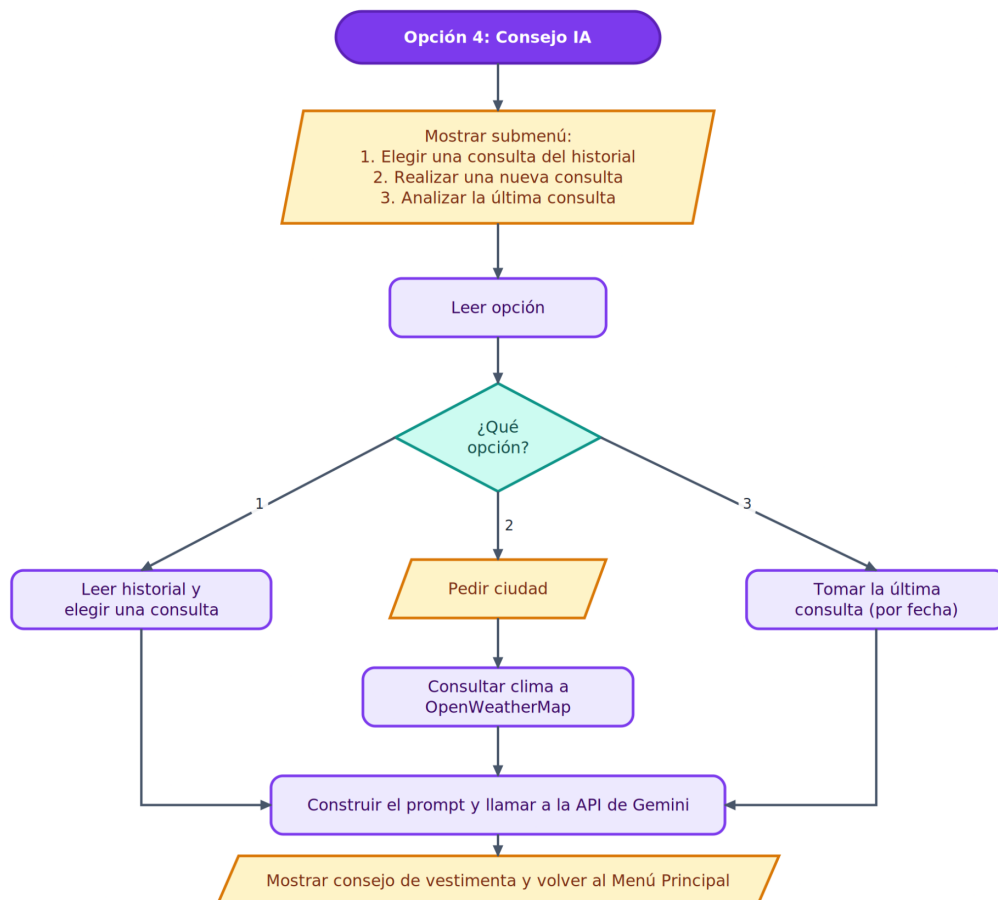
Opción 2: Ver historial del clima



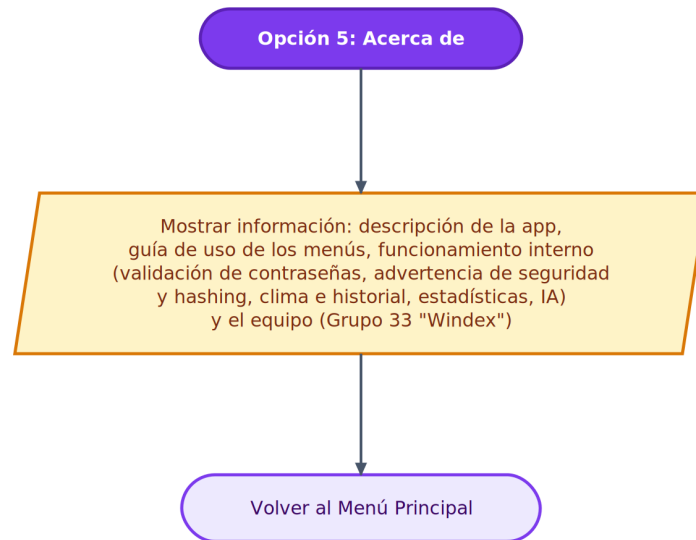
Opción 3: Estadísticas globales y exportar



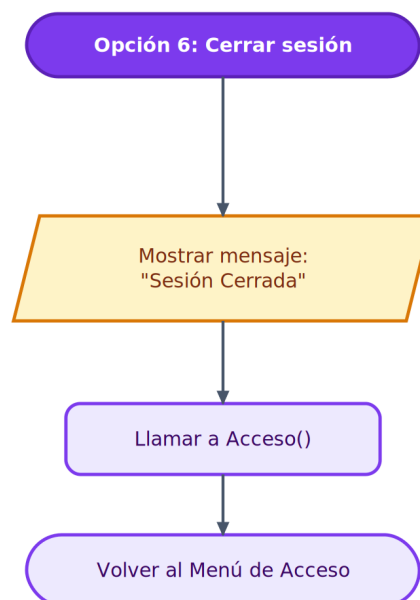
Opción 4: Consejo IA - ¿Cómo me visto hoy?



Opción 5: Acerca de



Opción 6: Cerrar sesión



3. Guía de Usuario Detallada

A continuación se explica cómo usar cada funcionalidad del Menú de Acceso y del Menú Principal.

Menú de Acceso: (1) Iniciar sesión solicita usuario y contraseña y los compara con usuarios.csv, respetando mayúsculas, minúsculas y caracteres especiales; si las credenciales son incorrectas, lo avisa y permite reintentar. (2) Registrar pide un nombre de usuario nuevo (verifica que no exista) y una contraseña que debe pasar la validación de seguridad: coincidir al repetirla, tener al menos 7 caracteres, al menos una mayúscula, al menos una minúscula y al menos un carácter especial; si no cumple, indica qué reglas incumplió y sugiere una contraseña más segura de ejemplo. (3) Salir cierra el programa.

Menú Principal: (1) Consultar clima pide una ciudad, muestra sus datos (temperatura, humedad, presión, viento y descripción) y los guarda en historial_global.csv. (2) Ver historial pide una ciudad y muestra las consultas registradas por el usuario logueado para esa ciudad. (3) Estadísticas globales y exportar abre un submenú que permite ver la ciudad más consultada, el total de consultas y la temperatura promedio, o bien exportar el historial completo a un archivo CSV. (4) Consejo IA abre un submenú con tres opciones (elegir una consulta del historial, hacer una nueva consulta o analizar la última consulta) y, con esos datos del clima, le pide a Gemini un consejo de vestimenta breve. (5) Acerca de muestra la información de la aplicación y del equipo. (6) Cerrar sesión vuelve al Menú de Acceso.

4. Desafíos y Soluciones

Los principales desafíos durante el desarrollo y cómo los resolvimos fueron los siguientes. Por un lado, la comparación entre la opción elegida en el menú y el valor esperado nos daba errores por lo que resolvimos unificando los tipos de datos (leyendo la opción como texto). También, tuvimos muchos problemas

cuando intentábamos leer archivos que quizá no existían por lo que tuvimos que enfocarnos en que siempre se esté leyendo el archivo correcto y saber cómo manejar los errores como el de `FileNotFoundException`. Para explicarle al usuario qué le faltaba a la contraseña: lo logramos armando una lista con las reglas incumplidas y mostrándolas todas juntas. Organizar las claves de API: las separamos del resto de la lógica en el módulo `api.py`, para no tenerlas dispersas por el código. Para organizar la lectura y escritura de los CSV de forma más cómoda, optamos por el uso de la librería de `pandas` debido a su cómoda sintaxis. Las importaciones circulares entre módulos (`main`, `auth` y `menu_principal`): las resolvimos importando algunas funciones de forma diferida, dentro de la función que las necesita. Y por último, diseñar un prompt de IA que devolviera un consejo breve y concreto lo armamos basándonos en el módulo de Prompt Engineering (2.7).

5. Conclusiones y Aprendizajes del Equipo.

Aprendimos a trabajar con APIs en la nube, comprobamos la utilidad de dividir el programa en módulos para encontrar y corregir errores, tomamos conciencia de por qué guardar contraseñas en archivos CSV es inseguro, vimos cómo un CSV bien organizado habilita el análisis de datos, e integramos Inteligencia Artificial entendiendo que la calidad de la respuesta depende del nivel de detallismo y precisión que demos en el prompt.